cocotb

Teams Members:

Ahmed Hossam Rafik
Ahmed Haitham Othman
Abdelrahman Khaled Fouad
Mohamed Ahmed Mohamed
Mohamed Adel Abdelrahem
Moustafa Saad Dawood
Youssef Mohamed Mamdouh

Supervision:

Dr. Maged Ghoneima

Dr. Ahmed Mehanna

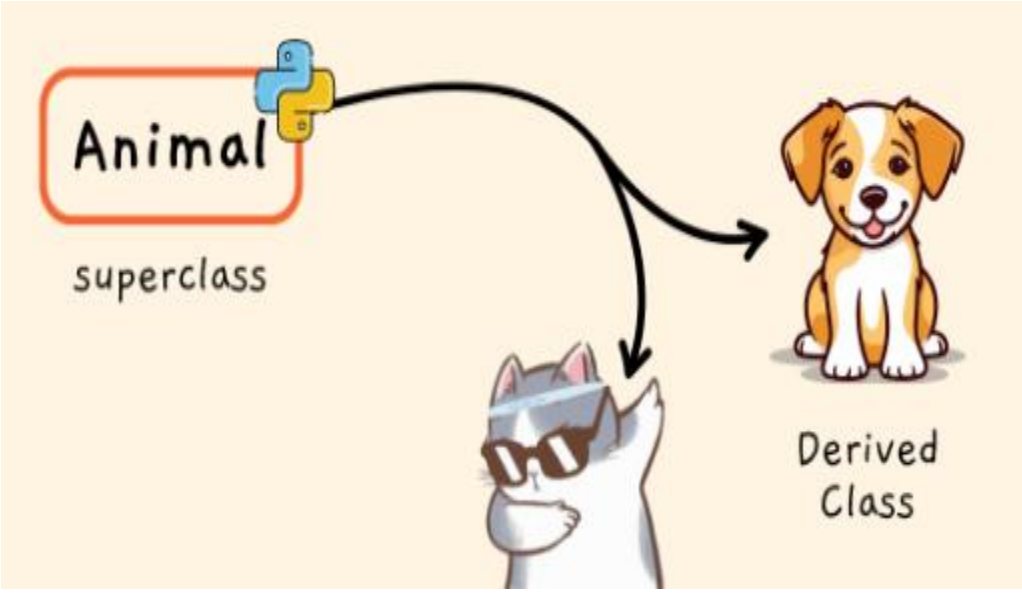
ADI Digital Team

Why Inheritance? (What NOT To Do)

Copying Code is Wrong

```python
class Dog:
    def make_sound(self):
        print("The dog says, 'bow bow'")

class Cat:
    def make_sound(self):
        print("The cat says, 'mião'")
```



🔴 **Problem:**

- Code duplication
- If we add new behavior (sleep, eat, etc.) → we write it twice
- Bugs must be fixed twice
- Poor maintainability

✅ **Solution:**

Use **Inheritance** to share common behavior.

Basic Inheritance (Single Inheritance)

1. Create a Base Class

```python
class Animal:
    def __init__(self):
        self.species = None
        self.sound = None

    def make_sound(self):
        print(f"The {self.species} says '{self.sound}'")
```

2. Extend It

```python
class Dog(Animal):
    def __init__(self):
        self.species = "dog"
        self.sound = "bow bow"

class Cat(Animal):
    def __init__(self):
        self.species = "cat"
        self.sound = "mião"
```

- class Dog(Animal) → Dog inherits from Animal
- *make_sound()* is inherited
- *__init__()* is overridden

How Python Inheritance Works

Important Rule:

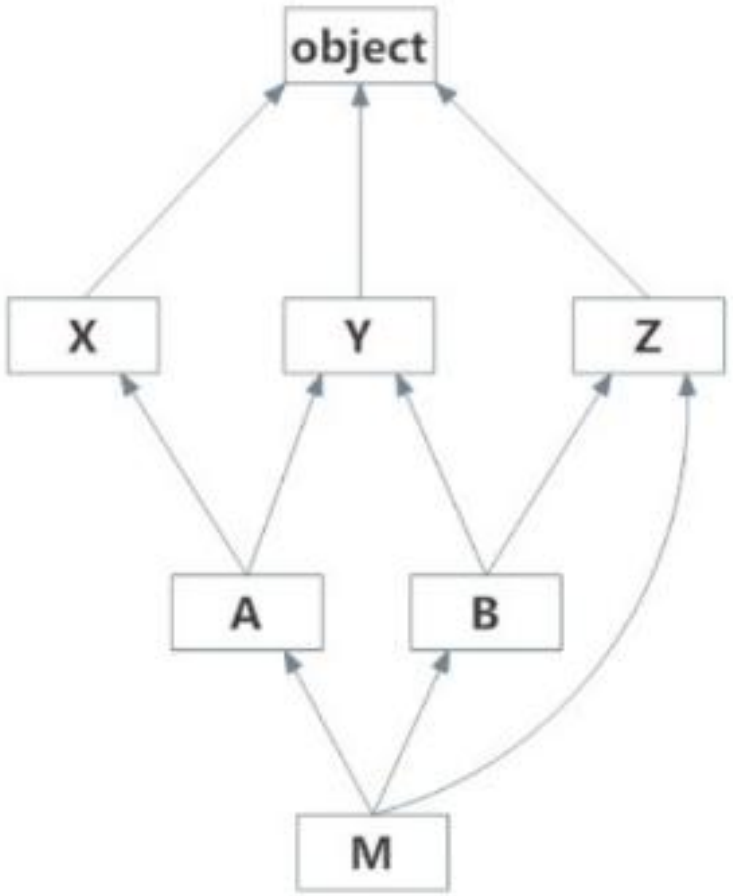
- Python **inherits methods only**
- It does NOT automatically inherit data attributes

When calling a method:

1. Python searches the object's class
2. If not found → searches parent
3. Continues using the **Method Resolution Order (MRO)**
4. If not found → error

```
print(SmallDog.mro())
```

search order: [SmallDog, Dog, Animal, object]



The Common Mistake (AttributeError)

```python
class SmallDog(Dog):
    def __init__(self):
        self.sound = "yap yap"

SmallDog().make_sound()
```

What Happens?

We get: AttributeError: 'SmallDog' object has no attribute 'species'

Why?

- *SmallDog.__init__()* did NOT call *parent __init__()*
- So self.species was never set

Even though Dog has species, **SmallDog must explicitly initialize it.**

Fixing It with super()

What *super()* Does:

- Looks at the MRO
- Calls the next class in the list
- Here → calls *Dog.__init__()*

Output: The dog says 'yap yap'

Correct Version:

```
class SmallDog(Dog):
    def __init__(self):
        super().__init__()
        self.sound = "yap yap"
```

✅ **Best Practice:** Always call *super().__init__()* when overriding *__init__()*.

Multiple Inheritance

Python supports **multiple inheritance.**

```python
class Parent(Human):
    def kiss(self):
        print(f"{self.name} gives the baby a kiss.")

class Firefighter(Human):
    def hose(self):
        print(f"{self.name} sprays water.")
```

```python
class FirefighterWithKids(Parent, Firefighter):
    pass
```

MRO()

print(FirefighterWithKids.mro())

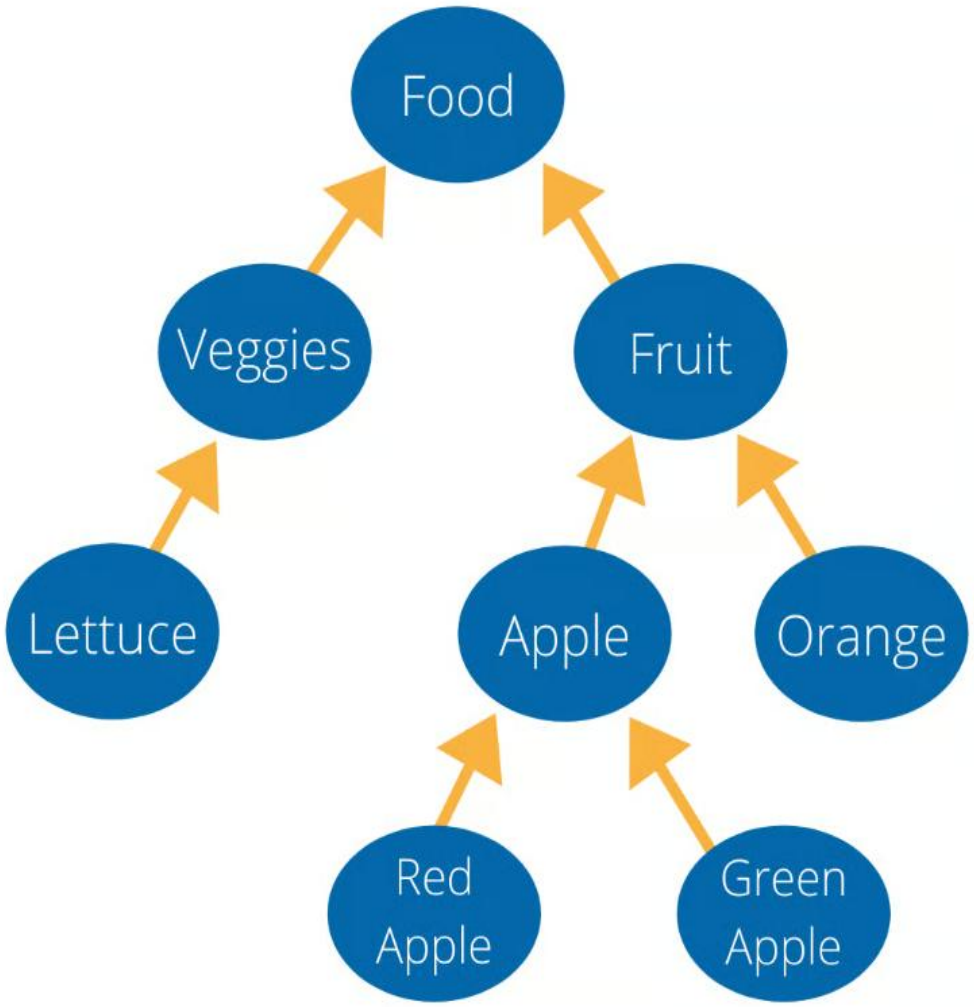
Search order:

1. FirefighterWithKids
2. Parent
3. Firefighter
4. Human
5. object

Order depends on parentheses order!

Summary

- Inheritance allows classes to reuse code by extending a base (parent) class instead of copying code.
- Python inheritance works by inheriting methods, while data attributes must be initialized explicitly in __init__().
- Python uses Method Resolution Order (MRO) to determine where a method is searched for in the class hierarchy.
- When overriding methods (especially __init__()), it is good practice to call the parent method using super().
- Python supports multiple inheritance, allowing a class to inherit behavior from more than one parent.
- In multiple inheritance designs, it is recommended to have only one __init__() responsible for initialization in the hierarchy.

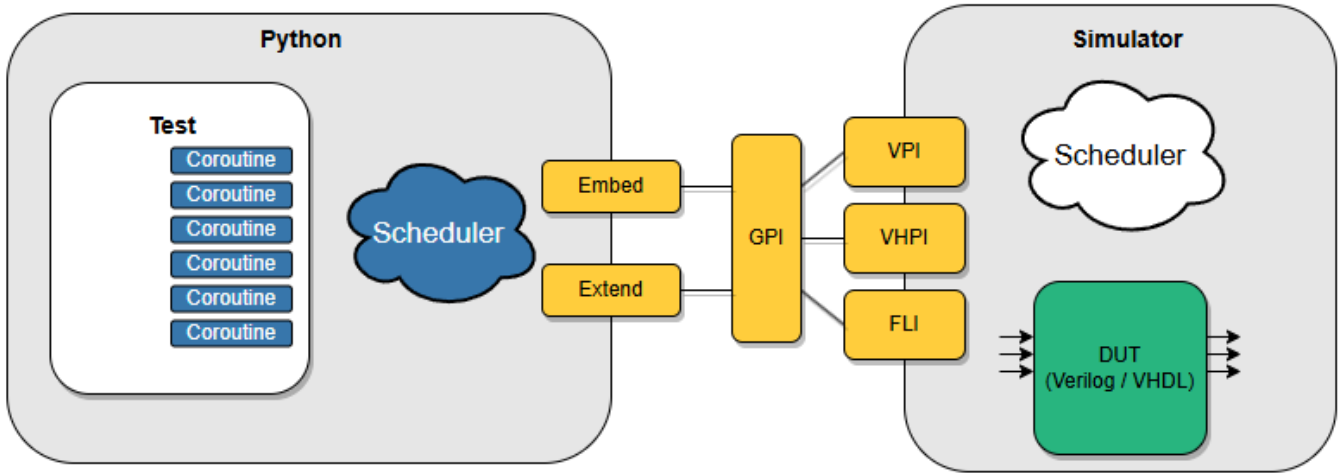


What is Cocotb?

- **Cocotb** is a *COroutine* based *COsimulation TestBench* environment for verifying VHDL and SystemVerilog RTL using Python
- How is cocotb different?
- All verification is done using Python which has various advantages over using SystemVerilog or VHDL for verification:
- Writing Python is **fast** - it's a very productive language.
- It's **easy** to interface to other languages from Python.
- Python has a huge library of existing code to **reuse**.
- Python is **interpreted** - tests can be edited and re-run without having to recompile the design.
- Python is **popular** - far more engineers know Python than SystemVerilog or VHDL

How does Cocotb work?

- **Direct Python Interfacing:** Drives stimulus and monitors HDL designs (Verilog/VHDL) directly from Python without requiring additional RTL wrapper code.
- **Standardized Connectivity:** Uses a Generic Peripheral Interface (GPI) to communicate with simulators through industry-standard APIs like VPI, VHPI, or FLI
- **Event-Driven Execution:** Synchronizes Python coroutines with the simulator using the **await** keyword to pass control back and forth as time advances



Creating a Test

- In cocotb, you can access all internals of your design, e.g. signals, ports, parameters, etc. through an object that is passed to each test. In the following we'll call this object(dut) key word

- This will first drive 10 periods of a square wave clock onto a port clk of toplevel
- After this, the clock stops, the value of my_signal_1 printed and the value of index [0] of my_signal_2 is checked to 0

```
import cocotb
from cocotb.triggers import Timer


@cocotb.test()
async def my_first_test(dut):
    """Try accessing the design."""

    for _ in range(10):
        dut.clk.value = 0
        await Timer(1, unit="ns")
        dut.clk.value = 1
        await Timer(1, unit="ns")

    cocotb.log.info("my_signal_1 is %s", dut.my_signal_1.value)
    assert dut.my_signal_2.value == 0
```

Things to note:

- Use the @cocotb.test() decorator to mark the test function to be run
- Use .value = value to assign a value to a signal.
- Use .value to get a signal's current value

Creating a Test

- The test shown is running sequentially, from start to end. Each await expression suspends execution of the test until whatever event the test is waiting for occurs and the simulator returns control back to cocotb
- It's most likely that you will want to do several things "at the same time"

  however - think multiple Always blocks in Verilog
- May move the clock generation part of the example above into its own async function

  and cocotb.start_soon() it ("start it in the background") from the test:

```python
import cocotb
from cocotb.triggers import Timer


@cocotb.test()
async def my_first_test(dut):
    """Try accessing the design."""

    for _ in range(10):
        dut.clk.value = 0
        await Timer(1, unit="ns")
        dut.clk.value = 1
        await Timer(1, unit="ns")

    cocotb.log.info("my_signal_1 is %s", dut.my_signal_1.value)
    assert dut.my_signal_2.value == 0
```

SV Mapping

```verilog
task automatic generate_clock();
    // "Generate clock pulses."

    repeat(10) begin
        clk = 0;
        #1ns;
        clk = 1;
        #1ns;
    end
endtask

initial begin // Equivalent to @cocotb.test() my_second_test
    // "Try accessing the design."

    fork
        generate_clock(); // run the clock "in the background"
    join_none

    #5ns; // wait a bit
    @(negedge clk); // wait for falling edge/"negedge"

    $info("my_signal_1 is %b", my_signal_1);
    assert(my_signal_2 == 0);
end
```

```python
# test_my_design.py (extended)

import cocotb
from cocotb.triggers import FallingEdge, Timer


async def generate_clock(dut):
    """Generate clock pulses."""

    for _ in range(10):
        dut.clk.value = 0
        await Timer(1, unit="ns")
        dut.clk.value = 1
        await Timer(1, unit="ns")


@cocotb.test()
async def my_second_test(dut):
    """Try accessing the design."""

    cocotb.start_soon(generate_clock(dut))  # run the clock "in the background"

    await Timer(5, unit="ns")  # wait a bit
    await FallingEdge(dut.clk)  # wait for falling edge/"negedge"

    cocotb.log.info("my_signal_1 is %s", dut.my_signal_1.value)
    assert dut.my_signal_2.value == 0
```

Logging

- Cocotb uses Python's logging lib with the configuration described in logging to provide some sensible defaults

So we use Cocotb.log.info instead of print functions or ` uvm_info

Note: Logging functions *only* log messages, they do not cause the test to fail.

```python
import logging
import cocotb

@cocotb.test()
async def test(dut):
    # Create a logger for this testbench
    logger = logging.getLogger("my_testbench")

    logger.debug("This is a debug message")
    logger.info("This is an info message")
    logger.warning("This is a warning message")
    logger.error("This is an error message")
    logger.critical("This is a critical message")
```

SystemVerilog (UVM)	Python (Cocotb)	Notes
`uvm_info("ID", "Msg", UVM_LOW)	logger.info("Msg")	Standard informational messages.
`uvm_info("ID", "Msg", UVM_HIGH)	logger.debug("Msg")	Used for deep, verbose debugging.
`uvm_warning("ID", "Msg")	logger.warning("Msg")	Highlights potential issues.
`uvm_error("ID", "Msg")	logger.error("Msg")	Logs an error (but test continues).
`uvm_fatal("ID", "Msg")	logger.critical("Msg")	Logs a critical failure (often stops the test).

Accessing the design

- When cocotb initializes it finds the toplevel instantiation in the simulator and creates a *handle* called dut. , Toplevel signals can be accessed using the “dot” notation used for accessing object attributes in Python

```
# Get a reference to the "clk" signal on the toplevel
clk = dut.clk

# Get a reference to a register "count"
# in a sub-block "inst_sub_block"
# (the instance name of a Verilog module or VHDL entity/component)
count = dut.inst_sub_block.count
```

- Assigning values to signals

Values can be assigned to signals using either the Value property of a handle object or set() method of a handle object.

- The assignment syntax sig.value=new_value

```
# Get a reference to the "clk" signal and assign a value
clk = dut.clk
clk.value = 1

# Direct assignment through the hierarchy
dut.input_signal.value = 12

# Assign a value to a memory deeper in the hierarchy
# ("inst_sub_block" and "inst_memory" are instance names of the
# respective Verilog modules or VHDL entity/components in the DUT)
dut.inst_sub_block.inst_memory.mem_array[4].value = 2
```

Managing Time: Sequential vs. Concurrent Execution

- **The Core Need:** In verification, testbenches must execute sequential steps (e.g., "wait for reset to drop, then start sending traffic") while simultaneously running background monitors and drivers.
- **The Python/Cocotb Solution:** This section teaches how to control whether a task **blocks** the main execution thread or runs in the **background**

The Three Rules of Execution

Sequential (Blocking): Using *await_task()* forces the test to wait until the task is 100% complete before moving to the next line of code. *(SV Equivalent: Calling a time-consuming task()*.

Concurrent (Non-Blocking): Wrapping a task in *cocotb.start_soon(task())* launches it in the background. The main test continues immediately. *(SV Equivalent: fork..join_none)*

Synchronization (Joining): Using *await task_handle* allows the main test to pause and wait for a specific background thread to finish. *(SV Equivalent: wait fork or waiting on a specific process)*.

```python
# A coroutine
async def reset_dut(reset_n, duration_ns):
    reset_n.value = 0
    await Timer(duration_ns, unit="ns")
    reset_n.value = 1
    cocotb.log.debug("Reset complete")

@cocotb.test()
async def parallel_example(dut):
    reset_n = dut.reset

    # Execution will block until reset_dut has completed
    await reset_dut(reset_n, 500)
    cocotb.log.debug("After reset")

    # Run reset_dut concurrently
    reset_thread = cocotb.start_soon(reset_dut(reset_n, duration_ns=500))

    # This timer will complete before the timer in the concurrently executing "reset_thread"
    await Timer(250, unit="ns")
    cocotb.log.debug("During reset (reset_n = %s)" % reset_n.value)

    # Wait for the other thread to complete
    await reset_thread
    cocotb.log.debug("After reset")
```

Forcing and freezing signals

- **Force:** Overrides all internal RTL drivers to lock a signal to a specific value (the exact equivalent of SV force
- **Release:** Removes a Force or Freeze condition, immediately returning control of the signal back to the RTL drivers (the exact equivalent of SV release)
- **Freeze:** Instantly locks a signal at its *current* existing value, ignoring all future RTL updates until explicitly released.
- **Deposit:** A standard procedural assignment (signal = vale) that drives a net until the RTL or another process overwrites it.

```
# Deposit action
dut.my_signal.value = 12
dut.my_signal.value = Deposit(12)  # equivalent syntax

# Force action
dut.my_signal.value = Force(12)    # my_signal stays 12 until released

# Release action
dut.my_signal.value = Release()    # Reverts any force/freeze assignments

# Freeze action
dut.my_signal.value = Freeze()     # my_signal stays at current value until released
```

Passing and failing tests

- When cocotb tests complete execution, they have either *passed* or *failed*
- In general, if a test coroutine completes without raising an *Exception* or if the test coroutine or any running Tasks calls cocotb.pass_test() the test is considered to have *passed*.
- **Natural Completion:** The test coroutine finishes executing completely without raising any Python *Exception*
- **Explicit Pass:** The main test coroutine, or any running background Task explicitly calls cocotb.pass_test()
- **Unhandled Cancellation:** The test raises a CancelledError (or awaits a cancelled Task without handling the error). The test will end immediately, but will still be marked as a **PASS**.

A passing test will print the following output.

```
0.00ns INFO     Test Passed: test
```

```python
@cocotb.test()
async def test(dut):
    assert 2 > 1  # assertion is correct, then the coroutine ends

@cocotb.test()
async def test(dut):
    cocotb.pass_test("Reason")  # ends test with success early
    assert 1 > 2  # this would fail, but it isn't run because the test was ended early

@cocotb.test()
async def test(dut):
    async def ends_test_with_pass():
        cocotb.pass_test("Reason")
    cocotb.start_soon(ends_test_with_pass())
    await Timer(10, 'ns')

@cocotb.test()
async def test(dut):
    async def cancelled_after_time():
        await Timer(1, unit='ns')
        raise CancelledError
    t = cocotb.start_soon(cancelled_after_time())
    await t
```

Passing and failing tests

- A Cocotb test is marked as a **FAIL** if the main test coroutine, or any background task it spawned, encounters an error. Specifically, this happens under three conditions.
- **Failed Assertions:** The primary way to intentionally fail a test. If a standard Python assert statement evaluates to False, it raises an AssertionError and fails the test
- **Unhandled Exceptions:** If your code triggers a standard Python error (such as a NameError for a typo, or an AttributeError for trying to drive a non-existent DUT signal) and you do not catch it, the test fails. **Note:** A CancelledError is the *only* exception that does not cause a failure.

Example

```
@cocotb.test()
async def test(dut):
    assert 1 > 2, "Testing the obvious"

@cocotb.test()
async def test(dut):
    async def fails_test():
        assert 1 > 2
    cocotb.start_soon(fails_test())
    await Timer(10, 'ns')
```

Output:

```
0.00ns ERROR     Test Failed: test (result was AssertionError)
                 Traceback (most recent call last):
                   File "test.py", line 3, in test
                     assert 1 > 2, "Testing the obvious"
                 AssertionError: Testing the obvious
```

Introduction to Coroutines and Tasks

Cocotb Concept:

- Testbenches use Python coroutines (**async def**).
- Simulation pauses when a coroutine uses await (e.g., **await Timer(10, 'ns') or await RisingEdge(clk))**, passing control back to the simulator.
- Coroutines can return values to be used by other coroutines.

SystemVerilog & UVM Relation:

- **SV Equivalent:** This is identical to the distinction between a function (executes in zero time) and a task (can consume simulation time).
- **UVM Equivalent:** UVM heavily relies on time-consuming tasks for stimulus generation, particularly in the **run_phase()** of UVM components and the body() task of UVM sequences. Awaiting a RisingEdge is exactly like **@(posedge clk)** in SV.

Example

```python
async def wait_for_edge(clk):
    # Pauses until the rising edge of the clock
    await RisingEdge(clk)
    cocotb.log.info("Edge detected!")

    # Pauses for 10 ns
    await Timer(10, unit='ns')
```

```verilog
task wait_for_edge(ref logic clk);
    // Pauses until the rising edge of the clock
    @(posedge clk);
    `uvm_info("TAG", "Edge detected!", UVM_LOW)

    // Pauses for 10 ns
    #10ns;
endtask
```

Output

```
10.00ns INFO     cocotb.test_name   Edge detected!
20.00ns INFO     cocotb.test_name   (Coroutine finishes after Timer)
```

```
UVM_INFO @ 10.00 ns: reporter [TAG] Edge detected!
# (Task finishes at 20.00 ns after the #10ns delay)
```

Concurrent Execution

Cocotb Concept:

- Coroutines can be scheduled to run concurrently in the background using **cocotb.start_soon().**
- These running coroutines are called Tasks. They execute independently, and the main task doesn't block unless it explicitly awaits them. Tasks can also be killed before completion.

SystemVerilog & UVM Relation:

- **SV Equivalent:** start_soon() is the direct equivalent of **fork ... join_none**. It spawns a new thread that runs in the background. Killing a task is similar to disable fork or using process control (process::kill()).
- **UVM Equivalent:** In UVM, parallel execution is common when starting multiple sequences on different sequencers simultaneously or running background **driver/monitor loops**.

Example

```python
@cocotb.test()
async def test_concurrent(dut):
    # Spawns a background task (runs independently)
    bg_task = cocotb.start_soon(monitor_signals(dut))

    # Main thread continues immediately
    await Timer(50, unit='ns')

    # Optionally kill the background task early
    bg_task.cancel()
```

```
task test_concurrent();
    process bg_task;

    // Spawns a background task (runs independently)
    fork
        begin
            bg_task = process::self();
            monitor_signals();
        end
    join_none

    // Main thread continues immediately
    #50ns;

    // Optionally kill the background task early
    if (bg_task != null) bg_task.kill();
endtask
```

Output

```
 0.00ns INFO     cocotb.test_concurrent   Test started, spawning monitor...
20.00ns INFO     cocotb.monitor_signals   Monitoring: data is valid
40.00ns INFO     cocotb.monitor_signals   Monitoring: data is valid
50.00ns INFO     cocotb.test_concurrent   Woke up after 50ns, cancelling monitor task
```

```
UVM_INFO @ 20.00 ns: reporter [MON] Monitoring: data is valid
UVM_INFO @ 40.00 ns: reporter [MON] Monitoring: data is valid
# At 50.00 ns, the main thread resumes and process::kill() silently terminates
```

Waiting for Multiple Events Simultaneously

Cocotb Concept:

- Cocotb provides objects like First and Combine to manage multiple running Tasks/Triggers.
- **Waiting For One Of Multiple Events (First)**: Resumes when the *first* of the passed triggers/tasks finishes. Commonly used for setting timeouts (e.g., wait for an event OR a timer).
- **Waiting For Multiple Events (Combine)**: Resumes only when *all* listed triggers/tasks finish.

SystemVerilog & UVM Relation:

- **SV Equivalent (First):** This is exactly how **fork … join_any** works. A very common SV pattern for timeouts is to fork an execution thread and a #timeout thread, using join_any to proceed whichever happens first.
- **SV Equivalent (Combine):** This maps precisely to **fork … join**, where the parent thread is blocked until all spawned threads complete.

Example

```python
# 1. Wait for ALL (Combine)
await Combine(
    cocotb.start_soon(task_A()),
    cocotb.start_soon(task_B())
)

# 2. Wait for FIRST / Timeout (First)
timeout = Timer(10, "us")
task = cocotb.start_soon(wait_for_packet())

result = await First(timeout, task)
if result is timeout:
    cocotb.log.error("Timeout occurred!")
```

```
// 1. Wait for ALL (Combine -> join)
fork
    task_A();
    task_B();
join

// 2. Wait for FIRST / Timeout (First -> join_any)
fork
    begin
        #10us; // Timeout thread
    end
    begin
        wait_for_packet(); // Actual task
    end
join_any
disable fork; // Kill the pending threads
```

Output

```
# --- Combine (Wait for All) ---
    200.00ns INFO     cocotb.test_name   Combine finished (waited for task_B to complete)

# --- First (Timeout scenario) ---
  10000.00ns ERROR    cocotb.test_name   Timeout occurred!
```

```
# --- Combine (fork ... join) ---
# Execution blocks until 200.00 ns when task_B finishes.

# --- First (fork ... join_any) ---
# The #10us delay finishes at 10000.00 ns.
# The 'disable fork' command silently kills the wait_for_packet() thread.
```

Async Generators

Cocotb Concept:

- Python allows yield statements inside an async def function, creating asynchronous generators.
- These can be iterated over time using async for, yielding a stream of values whenever a specific event occurs (e.g., returning a sample every clock cycle).

SystemVerilog & UVM Relation:

- **UVM Equivalent:** While SV doesn't have language-level "generators", this pattern is the functional equivalent of a **UVM Monitor** or a **UVM Sequence**.
  - A UVM Sequence generates a stream of uvm_sequence_items over time (**start_item -> finish_item**).
  - A UVM Monitor passively watches a bus and broadcasts observed transactions sequentially using an uvm_analysis_port.write(). Cocotb async generators provide a much more concise, Pythonic way to do this same stream generation.

return vs. yield

- **return is like hanging up the phone.** When a function hits a return, it gives back a value, destroys all of its local variables, and the function is completely finished. If you call it again, it starts over from the very beginning.
- **yield is like putting someone on hold.** When a function hits a yield, it gives back a value, but it *freezes* its current state. All the variables, loops, and data are kept alive in memory. The next time you ask that function for a value, it wakes up right where it was paused and continues executing until it hits the next yield.

Async Generators

Example

```python
# Async generator yielding data every clock cycle
async def data_monitor(clk, signal):
    while True:
        await RisingEdge(clk)
        yield signal.value

@cocotb.test()
async def test_stream(dut):
    # Consuming the stream
    async for data in data_monitor(dut.clk, dut.data_out):
        cocotb.log.info(f"Received: {data}")
```

```
// UVM Monitor acting as the generator
class my_monitor extends uvm_monitor;
    uvm_analysis_port #(logic [7:0]) ap;

    task run_phase(uvm_phase phase);
        forever begin
            @(posedge vif.clk);
            // Broadcasting the stream
            ap.write(vif.data_out);
        end
    endtask
endclass
```

Output

```
10.00ns INFO     cocotb.test_stream   Received: 165
20.00ns INFO     cocotb.test_stream   Received: 90
30.00ns INFO     cocotb.test_stream   Received: 255
```

```
UVM_INFO @ 10.00 ns: uvm_test_top.env.sb [SCOREBOARD] Received broadcast: 'ha5
UVM_INFO @ 20.00 ns: uvm_test_top.env.sb [SCOREBOARD] Received broadcast: 'h5a
UVM_INFO @ 30.00 ns: uvm_test_top.env.sb [SCOREBOARD] Received broadcast: 'hff
```

cocotb Queue

Starting a task would be of limited use if tasks could **not communicate**. So as we begin work on testbenches, we'll be creating tasks that share data. The Python Queue class allows task communication.

The cocotb extension of Queue enables this class to work within cocotb . We get the cocotb Queue by importing it from the module cocotb.queue .

```
# Figure 1: Importing cocotb Queue classes
from cocotb.queue import Queue, QueueEmpty, QueueFull
```

This import statement imports the cocotb version of the Queue alongside the QueueFull and QueueEmpty exceptions.

Since the tasks are running **in parallel**, we can't know whether the producer will send the data before the consumer reads it.

Queue objects solve this problem by making consumers wait until the producer has sent the data or forcing the producer to wait until the consumer is ready to receive the data.

When you have multiple tasks running at the same time—for instance, a Monitor capturing data from your hardware pins and a Scoreboard checking if that data is correct—they usually share information using a **Queue** (the Python equivalent of a UVM **uvm_tlm_fifo** or standard SystemVerilog **mailbox**).

We can handle the waiting between tasks in two ways. Either the task blocks until it can send or receive data, or the task polls the queue and waits on another coroutine before polling the queue again. The first is called **blocking communication** and the second is called, oddly enough, **nonblocking communication**.

cocotb Queue

1. Blocking Communication (The "Wait Right Here" Method)

- **The Concept:** The task goes to the queue and says, *"Give me the next piece of data. If the queue is empty, I will go to sleep right here and freeze this specific task until someone puts data in.“*

- **UVM Equivalent:** mailbox.get(data) or a standard TLM blocking get()

- **Hardware Analogy:** A state machine that halts all progress, constantly holding the read_enable high on an empty FIFO, refusing to do anything else until the empty flag drops.

```
# The scoreboard task freezes on this line until the monitor adds data
data = await my_queue.get()
print("Got data!")
```

cocotb Queue

2. Non-blocking Communication (The "Polling" Method)

- **The Concept:** The task goes to the queue and says, *"Do you have data right now? No? Okay, I'm not going to freeze. I will go do something else (like wait for the next clock cycle) and check back later."*
- **UVM Equivalent:** mailbox.try_get(data).

- **Hardware Analogy:** A state machine that checks a FIFO's empty flag. If it's empty, it moves on to service a different interface or bus, and checks the FIFO again on the next clock cycle.

```
from cocotb.queue import QueueEmpty

# Polling loop
while True:
    try:
        # Try to grab data instantly without awaiting/freezing
        data = my_queue.get_nowait()
        print("Got data!")
        break # Got it, exit the loop
    except QueueEmpty:
        # Queue was empty! Don't freeze. Wait 1 clock cycle and try again.
        await RisingEdge(dut.clk)
```

cocotb Queue

Blocking communication

Queues implement blocking communication with two coroutines named **put()** and **get()** . We'll demonstrate these with two coroutines named **Producer** and **Consumer** . The Producer sends numbers through a queue, and the consumer reads them from a queue.

```
async def Producer(queue, nn, delay=None) :
        """Produce numbers from 1 to nn and send them"""
        for datum in range(1, nn + 1) :
                if delay is not None:
        await Timer(delay, units="ns")
        await queue.put(datum)
        logger.info(f"Producer sent {datum}")
```

```
async def Consumer(queue) :
        """Get numbers and print them to the log"""
        while True:
                datum = await queue.get()
                logger.info(f"Consumer got {datum}")
```

The coroutine loops forever, getting numbers from the queue using **datum = await queue.get()** if the queue is empty, the Consumer loop blocks. It logs the times that it got the datum.

cocotb Queue

An infinitely long queue

The simplest way to communicate between tasks is to create an infinitely long queue. The producer writes its data into the queue, and then the consumer reads all the data in the same order.

The infinite Queue allows the Producer to run to completion, then hand off execution to the Consumer .

```
@cocotb.test()
async def infinite_queue(_) :
        """Show an infinite queue"""
        queue = Queue()
        cocotb.start_soon(Consumer(queue) )
        cocotb.start_soon(Producer(queue, 3) )
        await Timer(1, units="ns")
--
0ns INFO Producer sent 1
0ns INFO Producer sent 2
0ns INFO Producer sent 3
0ns INFO Consumer got 1
0ns INFO Consumer got 2
0ns INFO Consumer got 3
```

cocotb Queue

A Queue of size 1

If we want the producer to block until the consumer has processed the data, we set the queue maxsize to 1 . Setting the maxsize to 1 means the producer can put one number into the queue but blocks when it tries to put (,) the second number into the queue. The blocked put() returns after the consumer empties the queue. The code is the same but for Queue (maxsize =1).

```
@cocotb.test()
async def queue_max_size_1(_) :
        """Show producer and consumer with maxsize 1"""
        queue = Queue(maxsize=1)
        cocotb.start_soon(Consumer(queue) )
        cocotb.start_soon(Producer(queue, 3) )
        await Timer(1, units="ns")
--
1.00ns INFO Producer sent 1
1.00ns INFO Consumer got 1
1.00ns INFO Producer sent 2
1.00ns INFO Consumer got 2
1.00ns INFO Producer sent 3
1.00ns INFO Consumer got 3
```

cocotb Queue

Nonblocking communication

- Up until now, we used **blocking communication** (await queue.put() or await queue.get()). In blocking communication, if a queue is full or empty, the task is forced to freeze right on that line of code, no matter how long it takes.
- **But now in nonblocking communication,** Instead of the queue forcing your task to freeze, your task actively "polls" (checks) the queue. If the queue isn't ready, your task stays awake and gets to decide what to do next (like waiting for a specific timer or clock edge).

➢ **The Functions: put_nowait() and get_nowait()**

- The most important detail here is that these are standard functions, **not coroutines**.
- Because they are not coroutines, you do **not** use the await keyword in front of them.
- They execute instantly in 0 simulation time. They either succeed immediately, or they fail immediately by raising an exception (an error).

cocotb Queue

Nonblocking communication

```
async def ProducerNoWait(queue, nn, delay=None) :
        """Produce numbers from 1 to nn and send them"""
        for datum in range(1, nn + 1) :
                if delay is not None:
                        await Timer(delay, units="ns")
                while True:
                        try:
                                queue.put_nowait(datum)
                                break
                        except QueueFull:
                                logger.info("Queue Full, waiting 1ns")
                                await Timer(1, units="ns")
                logger.info(f"Producer sent {datum}")
```

```
async def ConsumerNoWait(queue) :
        """Get numbers and print them to the log"""
        while True:
                while True:
                        try:
                                datum = queue.get_nowait()
                                break
                        except QueueEmpty:
                                logger.info("Queue Empty, waiting 2 ns")
                                await Timer(2, units="ns")
                logger.info(f"Consumer got {datum}")
```

Why use this instead of blocking?

If you have a complex bus agent that needs to check a FIFO, but *also* needs to constantly monitor an interrupt pin, a blocking await queue.get() would freeze the agent completely, making it blind to the interrupt.

By using get_nowait(), the agent can quickly check the queue, see it's empty, and immediately go check the interrupt pin on the exact same clock cycle.

What Are Triggers in cocotb?

Definition

- **Triggers tell the cocotb scheduler when to resume a coroutine**
- A coroutine pauses at:

```
await <trigger>
```

- When the trigger fires → coroutine resumes

Example

```python
async def coro():
    print("Before clock edge")
    await RisingEdge(clk)
    print("After rising edge")
```

cocotb	SystemVerilog / UVM
await RisingEdge(clk)	@(posedge clk)
coroutine	task
cocotb scheduler	simulator event scheduler

Think of triggers as Python's version of @(..) event control !

Edge Triggers

These are hardware-event based triggers.

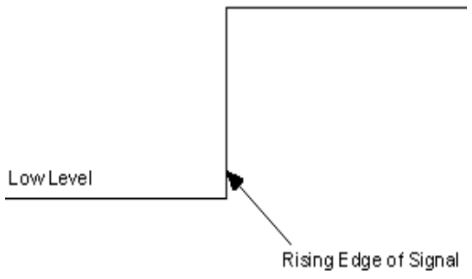
RisingEdge

```
await RisingEdge(clk)
```

- Fires on transition to 1
- Only for 1-bit signals

Preferred:

```
await clk.rising_edge
```



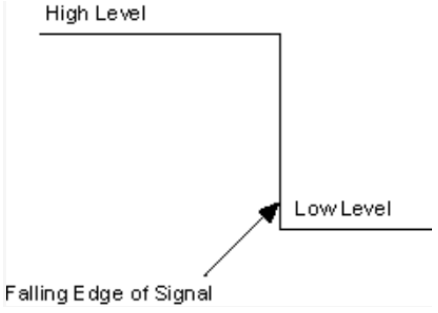
FallingEdge

```
await FallingEdge(clk)
```

- Fires on transition to 0
- Only for 1-bit signals

Preferred:

```
await clk.falling_edge
```



cocotb	SystemVerilog
RisingEdge(clk)	@(posedge clk)
FallingEdge(clk)	@(negedge clk)
ValueChange(sig)	@(sig)

ClockCycles Trigger

Wait for Multiple Cycles

```
await ClockCycles(clk, 5)
```

With explicit type:

```
await ClockCycles(clk, 5, edge_type=FallingEdge)
```

UVM Mapping

```
repeat (5) @(posedge clk);
```

Useful for:

- Reset sequences
- Delays between transactions
- Protocol timing gaps

Timer (Time-Based Trigger)

Timer:

```
await Timer(100, unit="ns")
```

Frequency Example

```
freq = 10e6
await Timer(1/freq, unit="sec")
```

UVM Mapping

```
#100ns;
```

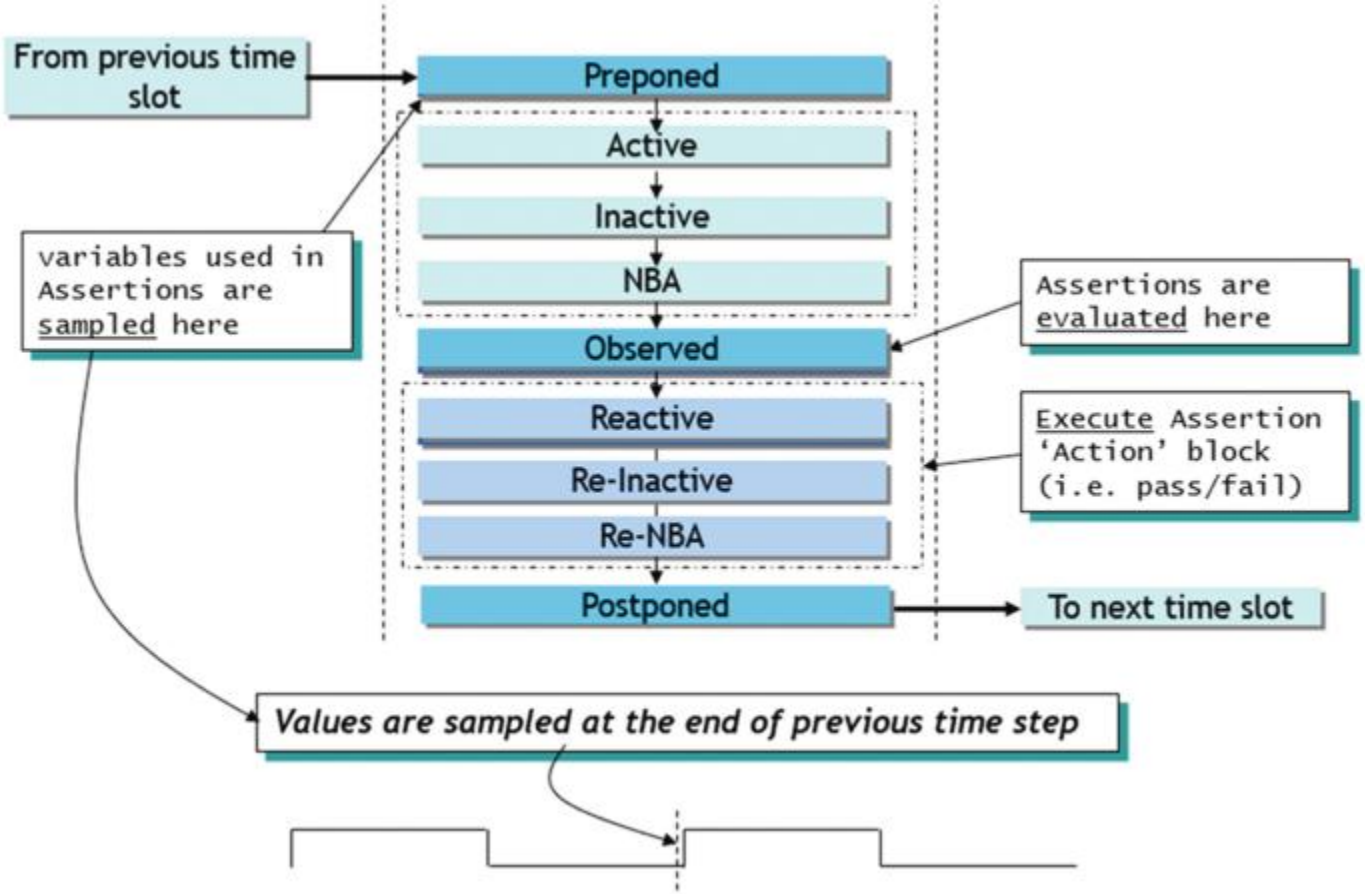
Important:

- Cannot pass 0 or negative
- Always consumes simulation time
- Resumes at beginning of next timestep

Simulation Phases Triggers

ReadOnly	ReadWrite	NextTimeStep
• Fires when signals are stable • No more delta cycles • Perfect for monitors	When simulator enters read-write phase	Move to next simulation timestep.

cocotb	SystemVerilog
ReadOnly	After all RTL updates
ReadWrite	Active region
NextTimeStep	#0 delay to next time slot



Especially important for race-condition avoidance.

Combine & First

Combine (wait for all)

```
await Combine(t1, t2)
```

UVM:

```
fork
  ...
join
```

First (wait for first)

```
result = await First(t1, t2)
```

UVM:

```
fork
  ...
join_any
```

Event (Synchronization)

Cocotb

```
e = Event()

await e.wait()
e.set()
```

UVM

```
uvm_event e = new("e");

task waiter();
    e.wait_trigger();
endtask

task trigger();
    e.trigger();
endtask
```

cocotb	UVM
Event()	uvm_event
e.wait()	wait_trigger()
e.set()	trigger()

Lock (Mutual Exclusion)

Guarantees fair scheduling.

Cocotb

```python
lock = Lock()

async with lock:
    # critical section
```

UVM

```perl
semaphore sem = new(1);
sem.get();
...
sem.put();
```

with_timeout

it's all about timing protection

Cocotb

```
await with_timeout(coro(), 100, "ns")
```

If timeout happens:
→ *SimTimeoutError* raised

Passed Object	On Timeout
coroutine	killed
task	continues running
trigger	cancelled

UVM

```
fork
   begin
      #100ns;
      timeout = 1;
   end
   begin
      transaction();
   end
join_any
disable fork;
```

Big Picture: Trigger Categories

Category	Examples	UVM Equivalent
Edge	RisingEdge	@(posedge)
Time	Timer	#delay
Multi-cycle	ClockCycles	repeat @
Concurrency	First, Combine	join_any, join
Sync	Event	uvm_event
Mutex	Lock	semaphore
Timeout	with_timeout	fork + timeout

Triggers are the **backbone** of time and event control in cocotb.
If UVM is event-driven via simulator regions,
then cocotb is event-driven via **awaitable triggers** managed by the Python scheduler.

Thank You!